sd04. Logging File System (LLD → HLD)

> First-hand 1p3a P0 (bank #4): "Design a Logging File System" — asked in an LLD/HLD-mixed round alongside LRU. Related retrieval coding: LC 635 (`1.coding-1.md`). JD tie-in: every OCI service owns its logging path; the fsync/loss-window discussion is the ops-excellence signal.

Index

1. Crux & why hard

- Naive: `log()` opens the file, writes the line, fsyncs → every log call costs a syscall + disk flush; the app is now as slow as its logger; one giant file can never be rotated/queried.
- Minimum primitives: **in-process buffer** (never block the app), **background flusher** (batch), **rotated segments** (bounded files, named by time), **time index** (range queries without scanning everything).
- 🗣️ "A logger is a tiny LSM: memory buffer → sealed immutable segments → query = binary search on segment time ranges."

2. Clarifying questions (assumed answers)

- Single process library, or a node-wide daemon? → start library, extend to daemon/central
- Loss tolerance on crash? → last ≤ 50 ms acceptable (state it!); fsync-per-line mode available for audit logs
- Query needs? → time-range scan, level filter; full-text search is out of scope (→ central store)
- Volume? → 10k lines/s × 200 B = 2 MB/s per node

3. FR / NFR

Req	Target	Enforced by
append() latency	p99 < 10 µs, never blocks on disk	ring buffer, drop policy
Durability window	≤ 50 ms loss on crash	timed group flush
Bounded disk	max N GB	size/time rotation + retention delete
Range query	binary search, not full scan	per-segment [t_min, t_max] index
Rotation	no lost lines during swap	swap-then-seal under lock

4. Back-of-envelope

- 2 MB/s → 64 MB segment every ~32 s; 7 d retention ≈ 1.2 TB/node uncompressed → compress sealed segments (~10×) ≈ 120 GB — so what: compression of *sealed* files is free (immutable), retention math drives the rotation size

5. LLD: writer path (key code)

```
import threading, time, queue

class Logger:
    def __init__(self, writer, cap=65536, flush_ms=50):
        self.q = queue.Queue(maxsize=cap)       # ring-ish: bounded
        self.dropped = 0
        self.writer = writer                    # SegmentWriter below
        self.flush_ms = flush_ms
        threading.Thread(target=self._drain, daemon=True).start()

    def log(self, level, msg):                  # hot path: enqueue only
        try:
            self.q.put_nowait((time.time(), level, msg))
        except queue.Full:
            self.dropped += 1                   # count, never block the app

    def _drain(self):
        batch = []
        while True:
            try:
                batch.append(self.q.get(timeout=self.flush_ms / 1000))
                while len(batch) < 1000:
                    batch.append(self.q.get_nowait())
            except queue.Empty:
                pass
            if batch:
                self.writer.append_batch(batch) # one write + one fsync per batch
                batch = []
# group commit: N lines amortize one fsync; loss window = flush_ms + batch in memory
```

```
class Logger {
    private final ArrayBlockingQueue<LogLine> q = new ArrayBlockingQueue<>(65536);
    private final AtomicLong dropped = new AtomicLong();

    void log(Level lvl, String msg) {                    // hot path
        if (!q.offer(new LogLine(System.currentTimeMillis(), lvl, msg)))
            dropped.incrementAndGet();                   // never block caller
    }

    void drainLoop(SegmentWriter w) throws InterruptedException {
        List<LogLine> batch = new ArrayList<>(1000);
        while (true) {
            LogLine first = q.poll(50, TimeUnit.MILLISECONDS);
            if (first != null) { batch.add(first); q.drainTo(batch, 999); }
            if (!batch.isEmpty()) { w.appendBatch(batch); batch.clear(); }
        }
    }
}
// offer() + drainTo() = lock-cheap producer/consumer; one fsync per batch
```

6. Segment rotation & retention

- Active segment `{stream}-{epoch_ms}.log`; roll when size ≥ 64 MB or age ≥ 5 min
- Roll = atomically swap writer to new file, then **seal** old (record t_min/t_max, compress, mark immutable)
- Retention: delete sealed segments past age/size budget — oldest first; audit streams exempt
- Levels: single stream with level field beats per-level files (per-level breaks time-ordered reads)

7. Query by time range (key code)

```
import bisect

def query(segments, t_from, t_to, level=None):
    # segments sorted by t_min; each: {t_min, t_max, path}
    starts = [s["t_min"] for s in segments]
    i = bisect.bisect_right(starts, t_from) - 1
    out = []
    for s in segments[max(i, 0):]:
        if s["t_min"] > t_to:
            break
        if s["t_max"] < t_from:
            continue
        for ts, lvl, msg in read_lines(s["path"]):      # scan only overlapping segs
            if t_from <= ts <= t_to and (level is None or lvl == level):
                out.append((ts, lvl, msg))
    return out
# binary search picks segments; scan cost ∝ overlap, not total history (LC 635 flavor)
```

8. Data model

```
CREATE TABLE segment_meta (
  stream   TEXT NOT NULL,
  path     TEXT NOT NULL,
  t_min    TIMESTAMPTZ NOT NULL,
  t_max    TIMESTAMPTZ,                 -- NULL while active
  size_b   BIGINT NOT NULL DEFAULT 0,
  sealed   BOOLEAN NOT NULL DEFAULT FALSE,
  PRIMARY KEY (stream, path)
);
CREATE INDEX seg_by_time ON segment_meta (stream, t_min);
-- line format: <epoch_ms> <LEVEL> <msg>\n  (the 1p3a log-parser coding question!)
```

9. HLD extension: ship to central store

- Sealed segments upload to object storage (sd17); a central indexer registers them → cross-node query = same binary-search over a global segment_meta
- Tail-shipping (live): the drain loop tees batches to Kafka (sd05 pipeline) for real-time search/alerting
- 🗣️ "Local durability first, central availability second — the node keeps logging through any central outage."

10. Failure modes

- Crash mid-batch → lose ≤ flush window; recovery scans last unsealed segment, truncates torn tail (length-prefixed records make torn writes detectable)
- Disk full → rotate fails: drop + count, alarm on `dropped > 0`; never crash the host app

- Clock jump backwards → segment t_min/t_max use max(seen); monotonic clock for intervals
- Hot query on active segment → read sealed copy of index + allow tail read with bounded scan

11. Trade-offs & defeaters

- Defeater: "fsync every line" — a 100-writes/s logger; group commit + stated loss window is the adult answer (audit mode = per-line fsync, opt-in, slow)
- Defeater: "one file forever" — unbounded, unqueryable, unrotatable
- Defeater: "block the app when the buffer is full" — logging must shed, not backpressure the business path; count drops
- mmap vs write(): write+fsync is simpler to reason about durability; say mmap exists

12. Pop-ups

- ❓ Multi-process on one host? → per-process files, or a local daemon over a Unix socket owning the segments
- ❓ Search by request-id? → that's the central indexer's job (inverted index); local design stays time-indexed
- ❓ Structured logs? → JSON lines change nothing structural; size grows ~2×
- **L5:** this is Kafka's storage layer in miniature — segments + index + retention (sd15 §storage)

13. Memorization card

- Hot path = bounded queue enqueue; background drain = batch + one fsync (group commit); loss window stated
- Segments: roll by 64 MB/5 min → seal → compress → retention delete; meta holds [t_min, t_max]
- Query: binary search segment index, scan overlaps only
- Drops counted, never block; torn-tail recovery via length-prefix